

State Space Representation

Control Engineering

Course Schedule

1. Systems
2. Continuous-time Controllers
3. Laplace Domain
4. Poles/Zeros & Stability
5. Controller Design
6. Discrete-time Controllers
7. State Space Representation
8. Advanced Control Methods

State Space Representation

- MIMO Systems
- State variables and state equations
- Controllability & Observability
- State-space control
- State estimation
- Pole Placement
- LQR
- Kalman filter

Multi-Input Multi-Output (MIMO) Systems

So far: single-input single-output (SISO) systems

Systems with multiple inputs and outputs (MIMO) become cumbersome to describe using transfer functions (focus on input–output behavior, limited insight into internal dynamics)



→ Use of state-space representation (explicit representation of internal system dynamics)

State Variables

State: minimal set of variables describing the system's memory

→ all information needed to predict future behavior

State variables define the system completely: given the current state and inputs, the future evolution is uniquely determined

- Often correspond to physical quantities, e.g., position, velocity, current, voltage, temperature
- Not unique: different choices of state variables can describe the same system

Input + Current State → Future State + Output
(instead of only relating input to output)

State Equations

State equation (system dynamics):

$$\dot{\vec{x}}(t) = A \vec{x}(t) + B \vec{u}(t)$$

Describes how the state evolves over time (time-domain formulation)

Output equation:

$$\vec{y}(t) = C \vec{x}(t) + D \vec{u}(t)$$

Describes how outputs are generated

$\vec{x}(t)$: state vector (internal system variables)

$\vec{u}(t)$: input vector

$\vec{y}(t)$: output vector

System matrices:

A : system matrix (dynamics)

B : input matrix

C : output matrix

D : feedthrough (direct input $\rightarrow$ output)

Compact matrix form: naturally extends to MIMO systems

Example: Mass-Spring-Damper System

Differential equation (Newton's law):

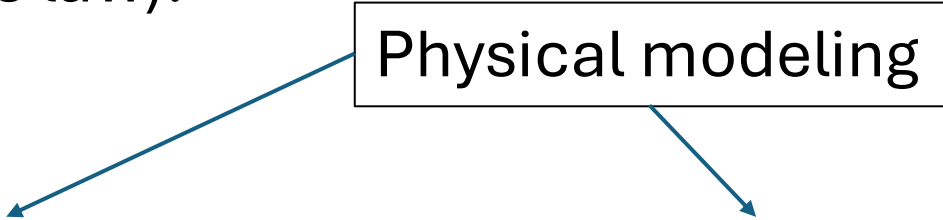
$$m\ddot{x} + \mu\dot{x} + kx = u(t)$$

Choice of state variables:

$$x_1(t) = x(t) \rightarrow \text{position}$$

$$x_2(t) = \dot{x}(t) \rightarrow \text{velocity}$$

Physical modeling



State-space form:

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = -\frac{k}{m}x_1 - \frac{\mu}{m}x_2 + \frac{1}{m}u$$

Matrix representation:

$$\dot{\vec{x}} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{\mu}{m} \end{bmatrix} \vec{x} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} \vec{u}$$

$$\vec{y} = [1 \quad 0] \vec{x}$$

For the sake of simplicity, we will no longer use vector arrows.

Physical Modeling or Construction

Two ways to get a state-space model:

1. From physics $\rightarrow$ define states $\rightarrow$ get matrices: physical states
 2. From a transfer function $\rightarrow$ define matrices $\rightarrow$ get states:
structured, useful for design (canonical forms)
- $\rightarrow$ Same system, just different coordinates

From Transfer Function to State Space

Transfer function describes input-output behavior in the Laplace domain:

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + \dots + b_1 s + b_0}{a_n s^n + \dots + a_1 s + a_0}$$

For controllable canonical form:

1. From $G(s)$, obtain time-domain equation (via inverse Laplace transform)
 2. Choose state variables based on derivatives of the output:
 $x_1 = y, \quad x_2 = \dot{y}, \quad \dots \quad x_n = y^{(n-1)}$
 3. Build state equations: Express highest derivative $y^{(n)}$ by lower-order states and input $u(t)$
 4. Assemble into standard matrix form: $\dot{x} = A x + B u, \quad y = C x + D u$
- Conversion of n th-order differential equation into system of n first-order equations

Example: Second-Order System

$$G(s) = \frac{1}{s^2 + 3s + 2}$$

1. $\ddot{y} + 3\dot{y} + 2y = u$

2. $x_1 = y, \quad x_2 = \dot{y}$ (not necessarily identical with the physical states)

3. $\dot{x}_1 = x_2, \quad \dot{x}_2 = -2x_1 - 3x_2 + u$

4. $\dot{x} = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u, \quad y = [1 \quad 0]x$

Controllable Canonical Form

Transfer function $\rightleftharpoons$ State space

→ same system, different representations

- Also multiple valid state-space models (equivalent representations of the same system): choice depends on analysis or design goals
- Canonical forms: designed system matrices (structured forms)
- Controllable canonical form: input influences all states step by step
- Example (second-order system, SISO): $G(s) = \frac{b_1s+b_0}{s^2+a_1s+a_0}$

$$A = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad C = [b_0 \quad b_1]$$

Observable Canonical Form

- State-space representation constructed to give observable structure
- Dual to controllable canonical form (transposed)
- Example (second-order system, SISO): $G(s) = \frac{b_1s+b_0}{s^2+a_1s+a_0}$

$$A = \begin{bmatrix} -a_1 & 1 \\ -a_0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} b_1 \\ b_0 \end{bmatrix}, \quad C = [1 \quad 0]$$


Output directly depends on first state

→ States for observable canonical form: $z_1 = y$, $z_2 = \dot{y} + 3y$
(typically not identical with the physical states)

Controllability & Observability

Controllability: Can we move the system anywhere we want using the input?

- Reaching all states via input signals
- If a state is not controllable: no input can influence it → cannot be stabilized
- Naturally emphasized in controllable canonical form

Observability: Can we figure out internal states from outputs?

- Diagnosing a system through measurements (outputs)
- If a state is not observable: it affects nothing we measure → hidden behavior
- Naturally emphasized in observable canonical form

→ A “good” system for control needs to be controllable + observable

State Feedback Control

Core idea: control input based on full state
(use all internal variables for feedback)

Typically:

$$x = x_{\text{real}} - x_{\text{desired}}$$

→ Shift coordinates to stabilize at $x \neq 0$

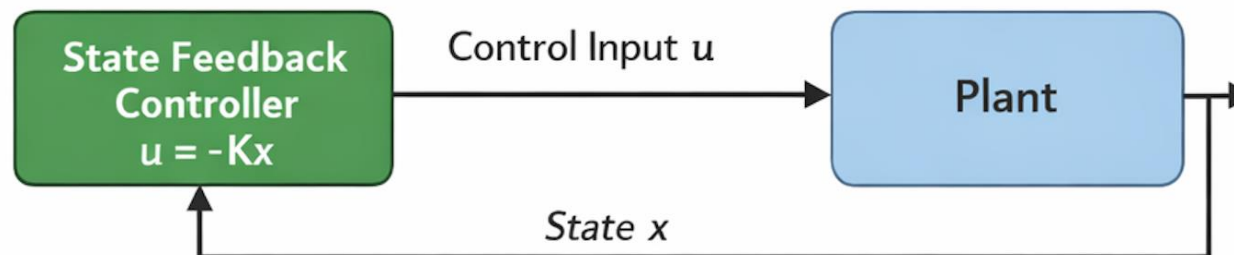
$$u(t) = -K x(t)$$

Feedback gains:

- For SISO K is a row vector
- For MIMO K is a full matrix

Goal: shape system dynamics

→ Faster response, improved stability, desired performance



Pole Placement

Deterministic (manual) way to design state feedback :

1. Choose desired closed-loop pole locations explicitly
2. Compute K such that eigenvalues of $(A - BK)$ match these locations

Requirement: system must be controllable

- Direct control over dynamics
- But no guidance on which poles are “best”

Matrix Exponential

Poles of transfer function (no feedback):

$$sX(s) = AX(s) + BU(s)$$

$$X(s) = (sI - A)^{-1}BU(s)$$

$$Y(s) = CX(s) + DU(s)$$

$$G(s) = \frac{Y(s)}{U(s)} = C(sI - A)^{-1}B + D$$

$$\rightarrow \text{poles } s: \det(sI - A) = 0$$

Insert $u = -Kx$ into $\dot{x} = Ax + Bu$

→ New system matrix of closed-loop system:

$$\dot{x} = \underbrace{(A - BK)}_{\text{Eigenvalues of this matrix}} x$$

($\det(A - BK - sI) = 0$)
correspond to closed-loop poles

Solution of the system:

$$x(t) = \underbrace{e^{(A - BK)t}}_{\text{State transition matrix}} x(0)$$

State transition matrix in form of matrix exponential (defined via power series):
evolution of a vector of coupled states

Linear Quadratic Regulator (LQR)

For a linear system: $\dot{x} = A x + B u$

In practice, using numerical solvers

find a state feedback: $u = -K x$

that minimizes the quadratic cost: $J = \int_0^\infty (x^T Q x + u^T R u) dt$

- $Q > 0$ penalizes state deviation
- $R > 0$ penalizes control effort

} Q and R chosen by the designer:
trade-off between performance and effort

- Optimal control: automatically computes “best” feedback gains
- Systematic design (no guesswork for poles)
- Robust and widely used in engineering
- Works naturally for MIMO systems

State Observers (State Estimation)

Issue: often not all state variables measurable

→ Need to reconstruct state from output → observers

Observer dynamics:

$$\dot{\hat{x}} = A\hat{x} + B u + L(y - C\hat{x})$$

observer gains

Key concept: estimate $\hat{x}(t) \approx x(t)$

Requirements: system must be observable

Enables state feedback with limited sensors

Output error: drives $\hat{x}(t) \rightarrow x(t)$

Estimation error $e = x - \hat{x}$

→ $\dot{e} = (A - LC)e$

→ Observer pole placement

Kalman Filter: Optimal State Estimation

Optimal observer for noisy systems

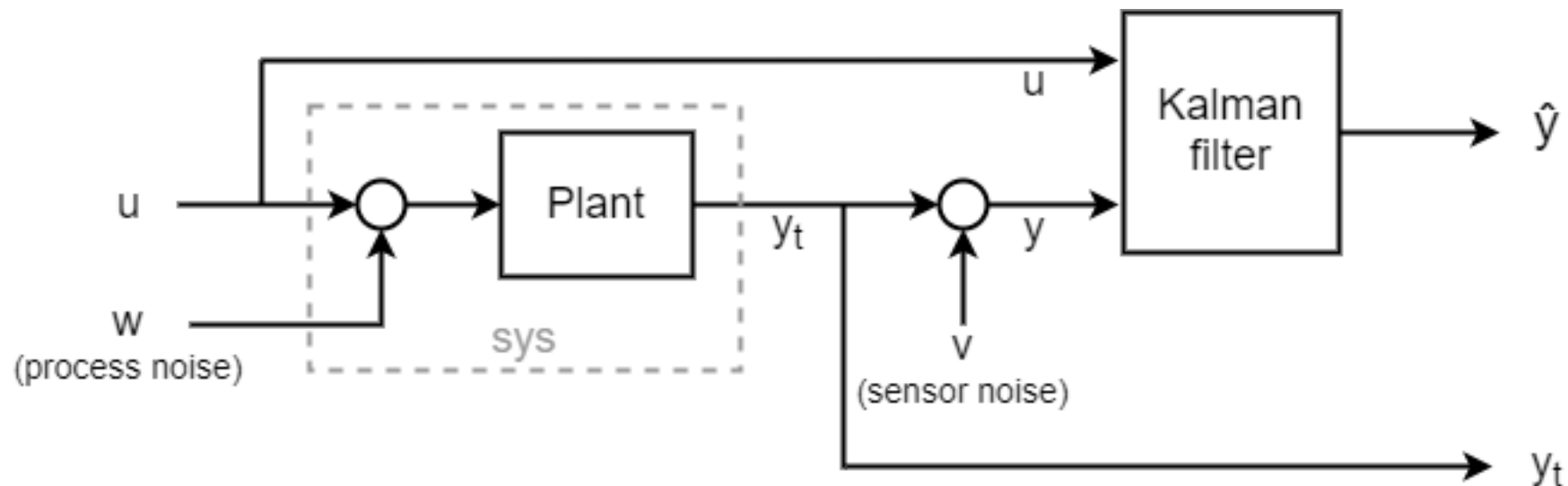
Handles measurement noise v and process noise w :

$$\dot{x} = Ax + Bu + w, \quad y = Cx + v$$

Same structure as general observer: $\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x})$

But chooses observer gain L optimally by minimizing the estimation error (analogous to pole placement and LQR)

Widely used in practice: robotics, aerospace, navigation, finance



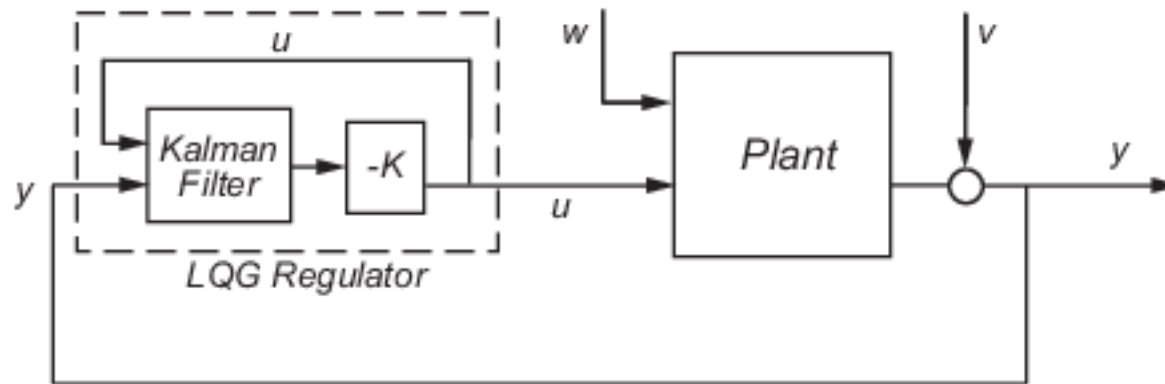
Linear Quadratic Gaussian (LQG) Control

Combination of both ideas (modern control architecture):

Observer (estimation) $\rightarrow \hat{x}(t) \rightarrow$ State Feedback (control)

Kalman filter: estimates system state from noisy measurements

LQR: computes optimal control input



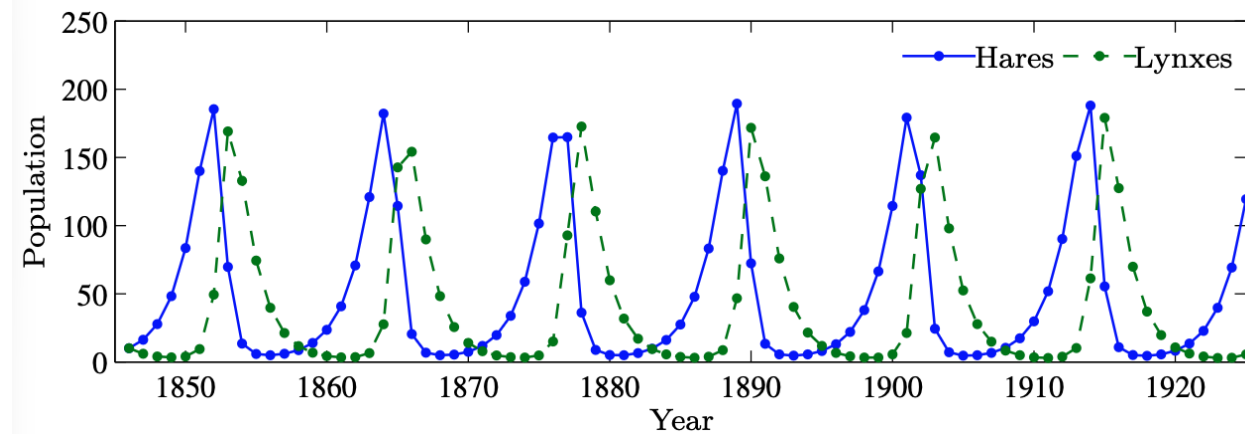
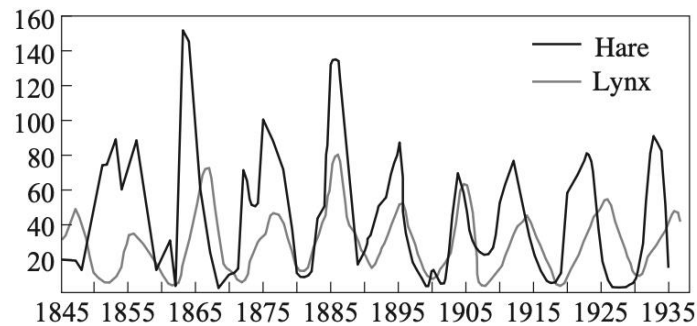
Separation principle: controller design (LQR) and estimator design (Kalman) can be done independently

Discrete-Time Note

Everything also exists in discrete time:

$$x_{k+1} = A x_k + B u_k$$

Example: predator-prey



Key Takeaways

- State space = internal system description
- Enables MIMO + modern control
- Controllability & observability are fundamental
- State feedback + observers → full control
- LQR & Kalman → optimal design